

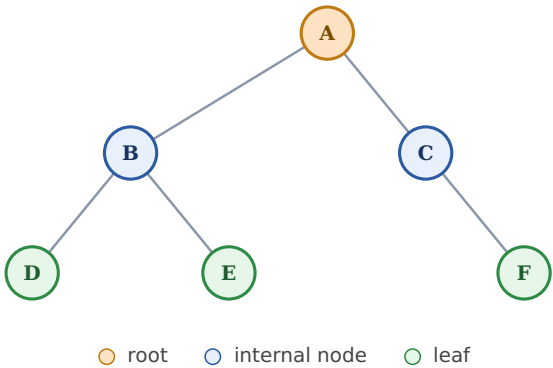
Binary Search Trees

Contents

- 1. Trees: basic terminology
- 2. Binary Trees
- 3. Height of a binary tree (best & worst case)
- 4. Binary Search Trees (BST): definition
- 5. Why insertion order changes the shape of a BST
- 6. BST as an Abstract Data Type
- 7. Search(node, x)
- 8. Insert(node, x)
- 9. Successor and Predecessor
- 10. Finding the minimum / maximum of a subtree
- 11. Delete(val) — all three cases, worked example
- 12. Complexity summary
- 13. Motivation: the Runway Reservation problem
- 14. Limitation of plain BSTs

1 Trees: basic terminology

A **tree** is a hierarchical data structure made of **nodes** connected by edges, with one special node called the **root**. Trees show up everywhere: an organization chart, a file system, a family tree.

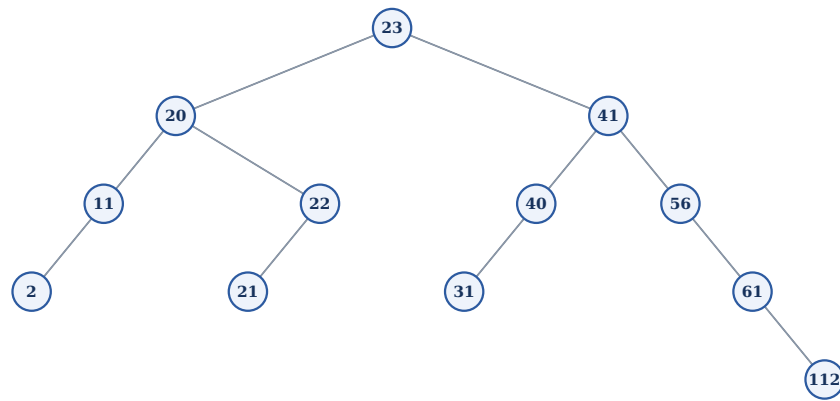


Term	Meaning
Root	The unique topmost node with no parent (here, A).
Parent / Child	A is the parent of B and C ; they are its children.
Ancestor / Descendant	A is an ancestor of D ; D is a descendant of A .
Sibling	Nodes sharing the same parent, e.g. D and E .
Leaf	A node with no children (D , E , F).
Internal node	Any node that is not a leaf (A , B , C).
Depth(x)	Number of edges from the root down to x . Depth(A)=0 , Depth(D)=2 .
Height of the tree	Length of the <i>longest</i> root-to-leaf path. Here, height = 2.

2 Binary Trees

Definition. A **binary tree** is an ordered tree in which every node has **at most 2 children**, conventionally called the *left child*

and the *right child*.



Example binary tree (this exact tree will also be our running BST example).

2.1 How a node is implemented

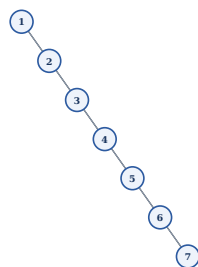
Just like a node in a linked list, every node of a binary tree stores:

- `val` — the data held at the node;
- `left` — pointer to the left child (or `NULL`);
- `right` — pointer to the right child (or `NULL`);
- `parent` — pointer back to the parent (or `NULL` for the root).

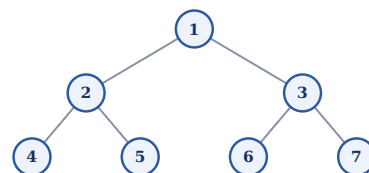
```
struct Node {
    int    val;
    Node*  left;
    Node*  right;
    Node*  parent;
};
```

3 Height of a binary tree — best & worst case

For a binary tree with `n` nodes, the height `h` can range between two extremes.



Maximum height: a pure chain (each node has one child).
 $h = n - 1$



Minimum height: as “bushy”/complete as possible.
 $h = \lfloor \log_2 n \rfloor$

General bound, for any binary tree of n nodes:

$$\lfloor \log_2 n \rfloor \leq h \leq n - 1$$

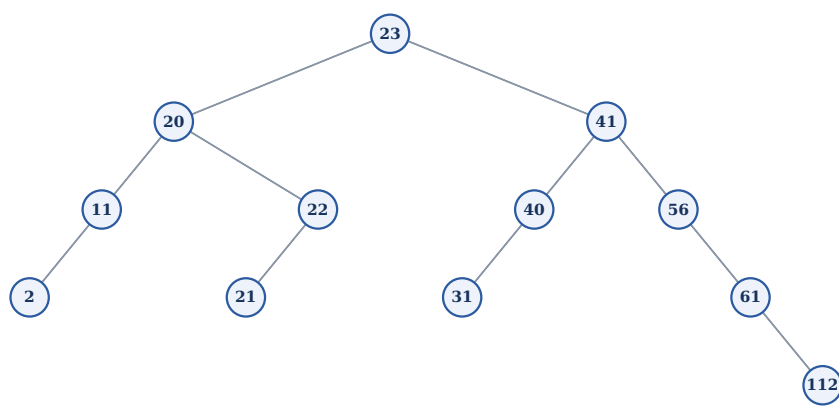
$$\text{Equivalently: } h + 1 \leq n \leq 2^{h+1} - 1$$

This bound is exactly why the height `h` matters so much: **every core BST operation runs in $O(h)$ time**, and h itself can be as small as $O(\log n)$ or as large as $O(n)$ depending on the tree's shape (more on this in Section 5).

4 Binary Search Trees (BST)

Definition. A **Binary Search Tree** is a binary tree that satisfies the **BST property**: for every node `x`,

- every value in the **left** subtree of `x` is **smaller** than `value(x)`;
- every value in the **right** subtree of `x` is **larger** than `value(x)`.

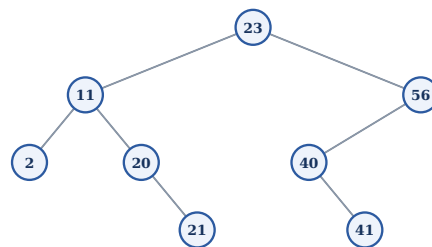


The same tree from Section 2 — check for yourself that the BST property holds at every node (e.g. everything under the left of 23 is <23 , everything under the right is >23 , and this repeats recursively).

5 Insertion order changes the shape of a BST

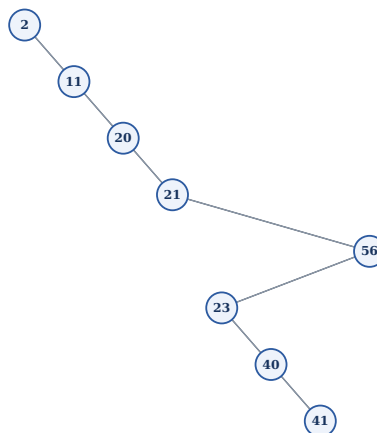
A BST is built by repeated calls to `Insert`. The **same set** of values, inserted in a **different order**, can produce very different-looking trees — and hence very different heights.

5.1 Sequence A: 23, 11, 20, 21, 2, 56, 40, 41



Inserting in this order gives a fairly balanced tree of height 3 for $n = 8$ nodes.

5.2 Sequence B: 2, 11, 20, 21, 56, 23, 40, 41 (same 8 values)



Because the first five values arrive in already-sorted order, the tree degenerates into a chain of height $7 = n-1$ — the *worst case* from Section 3.

Takeaway. Inserting already-sorted (or nearly sorted) data into a plain BST is the worst possible input: it produces a linked-list-shaped tree, and every operation degrades from $O(\log n)$ to $O(n)$.

6 BST as an Abstract Data Type (ADT)

A BST supports the following operations, all defined recursively on the BST property:

Operation	Purpose
<code>Search(node, x)</code>	Returns the node containing <code>x</code> , or <code>NULL</code> if absent.

`Insert(node, x)` Inserts `x` into the tree, preserving the BST property.

`Succ(val)` Returns the smallest element strictly greater than `val`.

`Pred(val)` Returns the largest element strictly smaller than `val`.

`Delete(val)` Removes `val` from the tree, preserving the BST property.

7 Search(node, x)

Because of the BST property, at every node we know which subtree could possibly contain `x` — so we throw away the other half of the tree at each step, just like binary search on a sorted array.

```
Search(node, x):
    if node == NULL:
        return NULL
    else if x == value(node):
        return node
    else if x < value(node):
        return Search(node.left, x)
    else:
        return Search(node.right, x)
```

Time complexity: $O(h)$ — we follow a single root-to-leaf path, at most. Best case $O(\log n)$ (balanced tree), worst case $O(n)$ (skewed tree).

8 Insert(node, x)

Insertion first does a `Search`-like walk to find where `x` belongs, then attaches a new leaf there.

```
Insert(node, x):
    if node == NULL:
        create a new node containing x, attach it to parent
    else if x < value(node):
        Insert(node.left, x)
    else if x > value(node):
        Insert(node.right, x)
    # (if x == value(node), most BST variants simply do nothing / disallow duplicates)
```

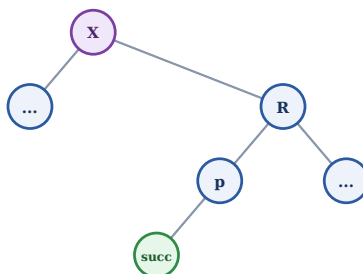
Time complexity: $O(h)$, same reasoning as `Search` — a new node always becomes a new leaf at the bottom of the path we walked.

9 Successor and Predecessor

`Succ(val)` returns the next larger element in sorted (in-order) sequence; `Pred(val)` returns the next smaller one. Finding the node that stores `val` first (via `Search`), there are exactly two cases for each.

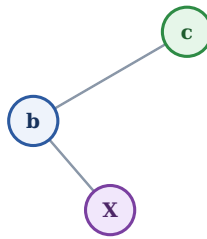
9.1 Successor — Case 1: node has a right child

The successor is the **smallest element of the right subtree** — keep walking left from `node.right` until there is no more left child.



9.2 Successor — Case 2: node has no right child

Climb up through ancestors starting at **x** until you reach a node that is a **left child** of its parent; that **parent** is the successor. (If we reach the root without ever being a left child, **x** was the maximum element — it has no successor.)



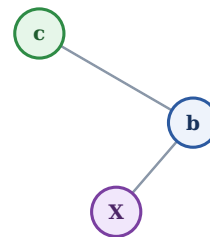
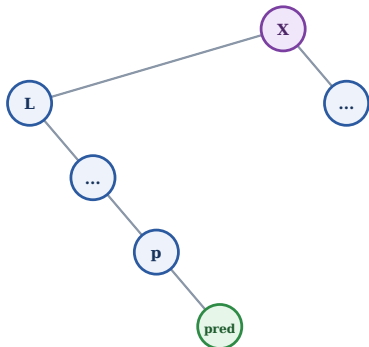
X has no right child. X is the right child of b, and b is the left child of c $\Rightarrow$ successor of X is c.

```

Succ(x):
    # x is a pointer to the node
    if x.right != NULL:
        return Min(x.right)      # Case 1
    temp1 = x
    temp2 = parent(x)
    while temp2 != NULL and temp1 == temp2.right:
        temp1 = temp2
        temp2 = parent(temp2)
    return temp2                  # Case 2 (NULL if x was the maximum)
  
```

9.3 Predecessor (symmetric)

Exactly the mirror image: swap every “left” with “right”.



Case 2: climb until X is a right child; that parent (c) is the predecessor.

Case 1: predecessor = largest (rightmost) node of the left subtree.

```

Pred(x):
    if x.left != NULL:
        return Max(x.left)      # Case 1
    temp1 = x
    temp2 = parent(x)
    while temp2 != NULL and temp1 == temp2.left:
        temp1 = temp2
        temp2 = parent(temp2)
    return temp2                  # Case 2
  
```

Time complexity: $O(h)$ for both — Case 1 walks down one root- to-leaf path, Case 2 walks up one path; either way the work is bounded by the height of the tree.

10 Finding the minimum / maximum of a (sub)tree

These are needed as helpers inside **Succ** , **Pred** , and (below) **Delete** .

```

Min(x):
    while x.left != NULL:
        x = x.left
    return x

Max(x):
    while x.right != NULL:
        x = x.right
    return x

```

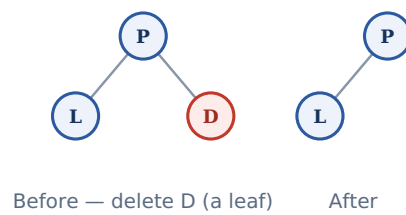
To find the smallest value in the subtree rooted at `x`, keep following left-child pointers until a node with no left child is reached — that node holds the minimum. Symmetrically, following right-child pointers gives the maximum. Both run in **O(h)** time.

11 Delete(val)

Deletion is the trickiest BST operation because removing a node must not break the BST property. First locate the node to delete (call it `node`), then handle **three cases** based on how many children it has.

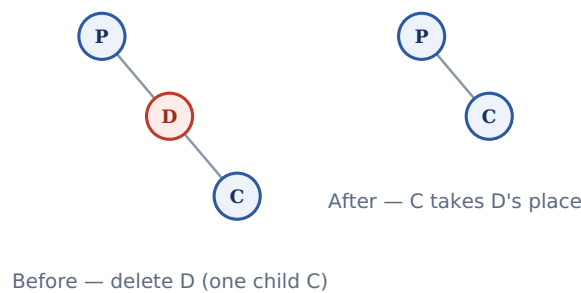
11.1 Case 1 — node has 0 children (leaf)

Trivial: just detach it from its parent.



11.2 Case 2 — node has 1 child (splice)

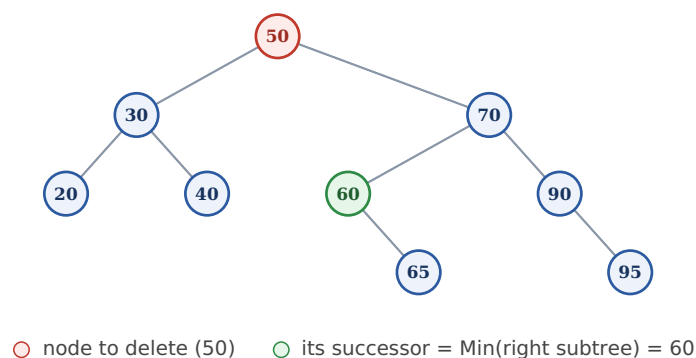
Connect the node's parent directly to its one child, “splicing” the deleted node out of the path.



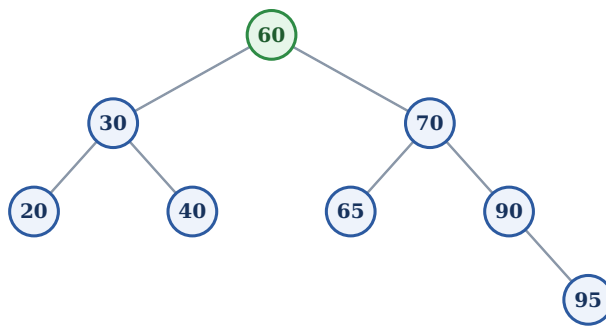
11.3 Case 3 — node has 2 children

- Find the **successor** node `x` of `node` (i.e. `Min(node.right)`) — by construction `x` has **no left child**, so it falls under Case 1 or Case 2.
- Splice** `x` out of the tree (recursively delete it using Case 1/2 above).
- Copy** `x`'s value into `node` (overwriting `val`).

Worked example — Delete(50)



50 has two children, so we find its successor: the smallest value in 50's right subtree is **60** (walk left from 70 → 60, which has no left child). Node 60 itself has a right child (65) but no left child, so removing 60 is a Case 2 splice: 65 takes 60's place. Finally we overwrite the value 50 with 60 at the root.



Result: root now holds 60; node 65 was spliced up into old 60's position.

Delete — pseudocode

```

Delete(val):
    node = Search(root, val)
    if node == NULL: return                # not found

    if node.left == NULL and node.right == NULL: # Case 1: leaf
        detach node from its parent (set the matching child pointer to NULL)

    else if node.left == NULL or node.right == NULL: # Case 2: one child
        child = node.left if node.left != NULL else node.right
        splice child into node's position (reattach to node.parent)

    else:                                     # Case 3: two children
        succ = Min(node.right)                # succ has no left child
        Delete(succ.val)                      # recursively remove succ (Case 1 or 2)
        node.val = succ.val                  # copy value up

```

Time complexity: $O(h)$ overall — locating the node is $O(h)$, and in the two-children case, finding the successor (`Min`) and splicing it out are each $O(h)$ as well (bounded by the same root-to-leaf path length).

12 Complexity summary

Operation	Time (general)	Best case (balanced, $h = \Theta(\log n)$)	Worst case (skewed, $h = \Theta(n)$)
<code>Search</code>	$O(h)$	$O(\log n)$	$O(n)$
<code>Insert</code>	$O(h)$	$O(\log n)$	$O(n)$
<code>Succ</code> / <code>Pred</code>	$O(h)$	$O(\log n)$	$O(n)$
<code>Min</code> / <code>Max</code>	$O(h)$	$O(\log n)$	$O(n)$
<code>Delete</code>	$O(h)$	$O(\log n)$	$O(n)$

Every single BST operation reduces to $O(h)$, and since $\lceil \log_2 n \rceil \leq h \leq n-1$ (Section 3), **all operations lie between $O(\log n)$ and $O(n)$** depending purely on the shape of the tree — which, as Section 5 showed, depends on the order elements were inserted.

13 Motivation: the Runway Reservation problem

Why do we want a structure like a BST at all? Consider a busy airport with a single runway. Landing requests arrive for a future time `t`; a request is approved only if no other approved landing is within `k` minutes of `t`. Once approved, `t` is added to the set `R` of reservations; it is removed once the plane has landed. With `|R| = n`, we would like **every** operation (search a neighbourhood of `t`, insert `t`, delete `t`) to run in $O(\log n)$ time.

Structure	Search	Insert	Comment
Unsorted list / array	$O(n)$	$O(1)$	Almost every operation is $O(n)$ except insertion.
Sorted array	$O(\log n)$	$O(n)$	Fast search (binary search), but insertion needs shifting elements.

Sorted linked list	$O(n)$	$O(1)$	Fast insertion once the spot is known, but finding that spot is $O(n)$.
Hash table	$O(1)$ avg	$O(1)$ avg	No notion of order — can't efficiently ask “is there a reservation within k minutes?”
Heap	—	$O(\log n)$	Great for min/max, but not good for searching an arbitrary range/neighbourhood.
Binary Search Tree	$O(h)$	$O(h)$	Keeps elements sorted <i>and</i> supports fast search, insert, delete, and neighbour queries (via Succ/Pred) — all from the same structure.

This is exactly the gap a BST fills: it behaves like a *sorted, dynamic* array — fast insertion **and** fast search, using Succ/Pred to check the k-minute neighbourhood of a candidate time t .

14 Limitation of plain BSTs

The one catch, demonstrated concretely in Section 5: a BST's performance is only as good as its shape. One natural way to measure how “balanced” a tree is: look at the difference between the **longest** root-to-leaf path and the **shortest** root-to-leaf path. If this gap is allowed to grow, height can approach $n - 1$ and every operation degrades to $O(n)$ — as bad as an unsorted list.

Keeping this gap small — i.e. **keeping the tree balanced** — is the whole motivation for self-balancing BSTs (e.g. AVL trees, red-black trees, using rotations) that guarantee $O(\log n)$ height at all times. That machinery is **beyond the scope of these notes**; here we only needed to see *why* it will eventually be necessary.